



DATA713 – MLOPS

SOUTENANCE DE PROJET :

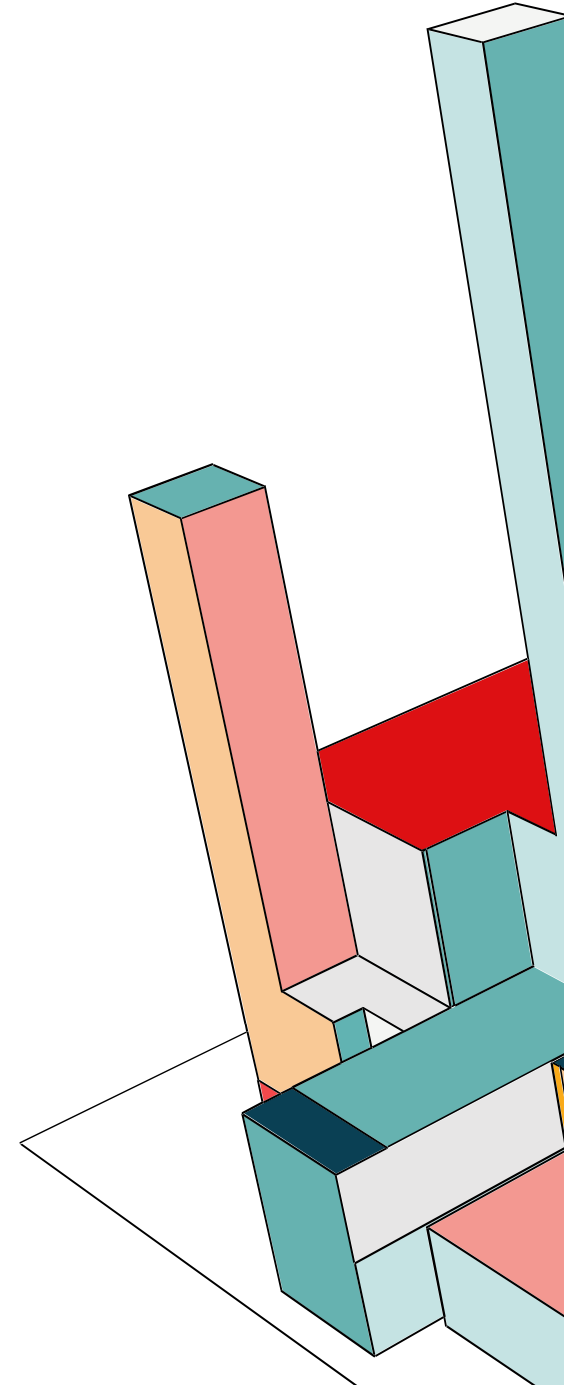
PLATEFORME MLOPS END-TO-END POUR
LA PRÉDICTION DE PRIX IMMOBILIER

TELECOM
Paris



SOMMAIRE

- I. Introduction et contexte métier
- II. Architecture globale
- III. Prétraitement des données et Machine Learning
- IV. Pipeline d'Entraînement (Continuous Training)
- V. Serving : API & Web App
- VI. Déploiement
- VII. Monitoring
- VIII. Démonstration

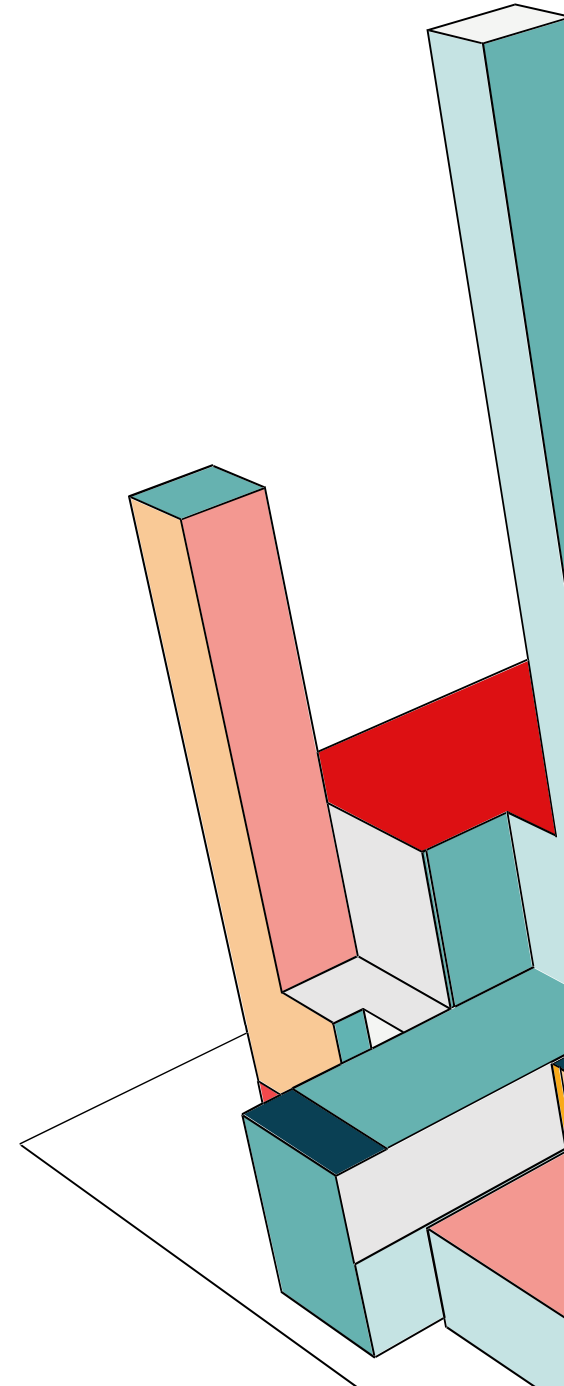


I. INTRODUCTION ET CONTEXTE MÉTIER



I. INTRODUCTION ET CONTEXTE MÉTIER

- **Objectif** : Plateforme MLOps end-to-end pour la prédiction de prix immobiliers (King County, WA)
- **Données** : 21 613 transactions, 21 features, Kaggle (Mai 2014 - Mai 2015)
- **Enjeux** : Couvrir les objectifs du cours





II. ARCHITECTURE GLOBALE

II. ARCHITECTURE GLOBALE

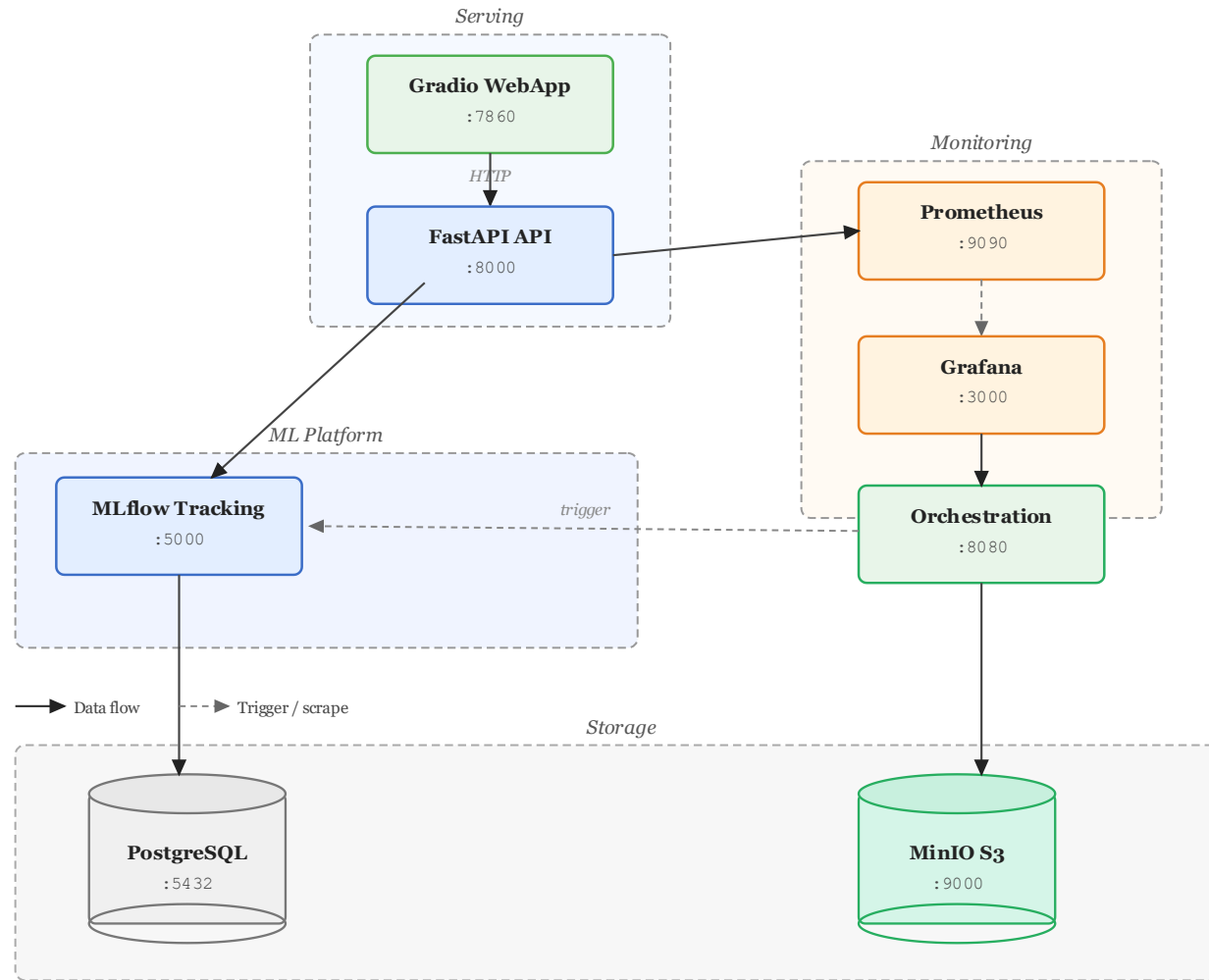
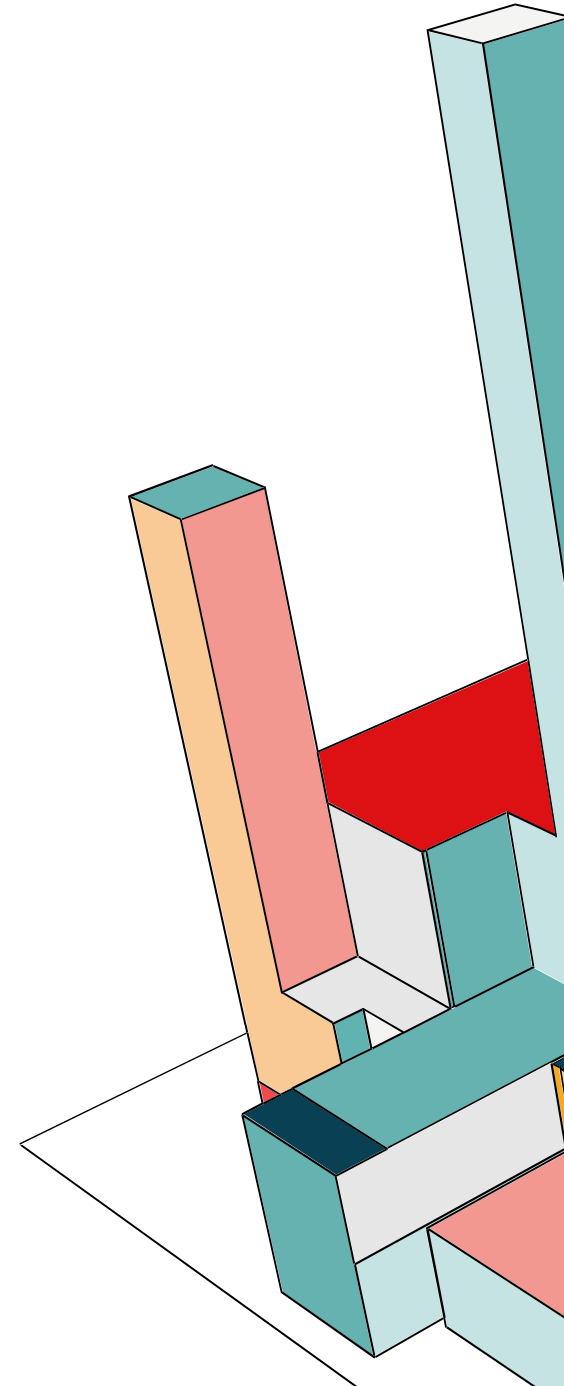


Figure 1. System architecture overview — ML serving, monitoring, platform & storage layers.



III. PRÉTRAITEMENT DE DONNÉES ET MACHINE LEARNING

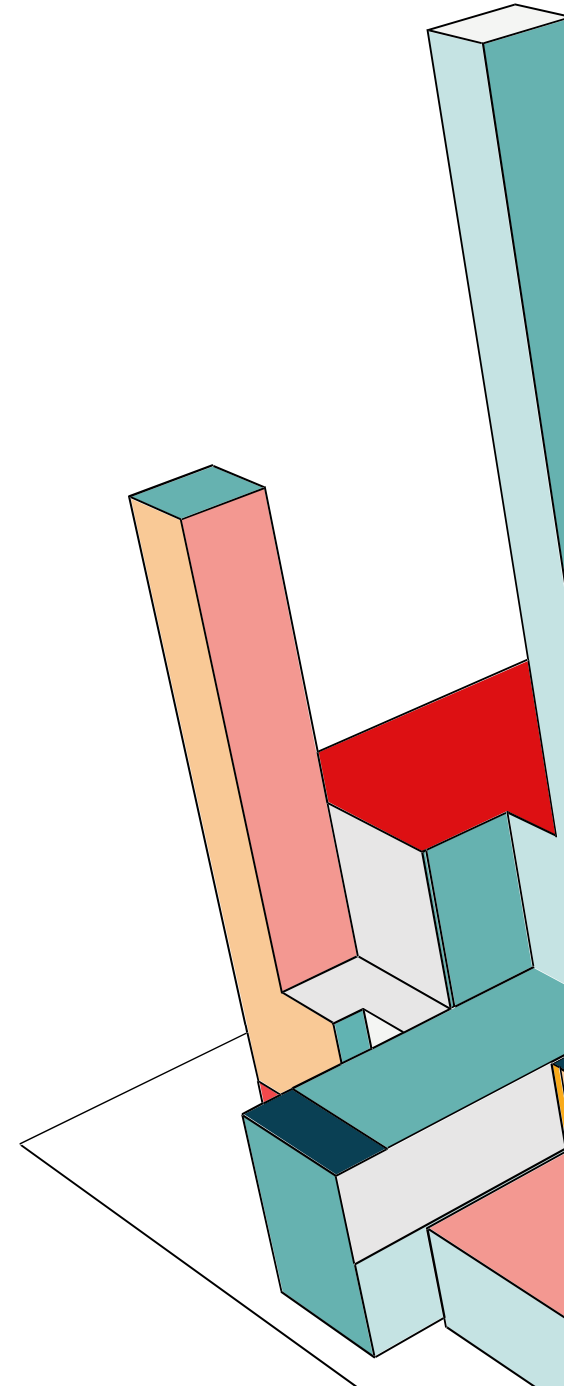


III. PIPELINE DE DONNÉES ET MACHINE LEARNING

- Prétraitement
 - Suppression d'outliers
 - Imputation de valeurs manquantes
 - Feature engineering (house_age, is_renovated, ...)
 - Encoding du code postal
- Modèle : XGBoost
 - Recherche d'hyperparamètres avec Optuna
 - 4 métriques d'évaluations : RMSE, MAE, R^2 , MAPE

$$\text{RMSE} = \left(\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \right)^{1/2} \quad \text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad \text{MAPE} = \frac{100}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$





IV. PIPELINE D'ENTRAÎNEMENT (CONTINUOUS TRAINING)

IV. PIPELINE D'ENTRAÎNEMENT (CONTINUOUS TRAINING)

DAGs

data_pipeline_dag.py

1. Vérifie si les données sont "fraîches".
2. Télécharge depuis Kaggle si nécessaire.
3. Valide les données : vérifie le nombre de colonnes/ lignes/ etc.
4. Preprocess et stocke les données.
5. Lance l'entraînement (second DAG).

training_dag.py

1. Vérifie l'existence des données.
2. Entraîne le modèle et logue dans mlflow : le modèle, les métriques, les paramètres et les artefacts.
3. Informe par mail en cas de succès.

Scripts utilisés par les DAGs

download.py

preprocess.py : nettoie les données (formattage, suppression de colonnes inutiles, engineering, ...)

train.py: entraînement XGBoost, Optuna, et sauvegarde dans mlflow (tracking - registry).

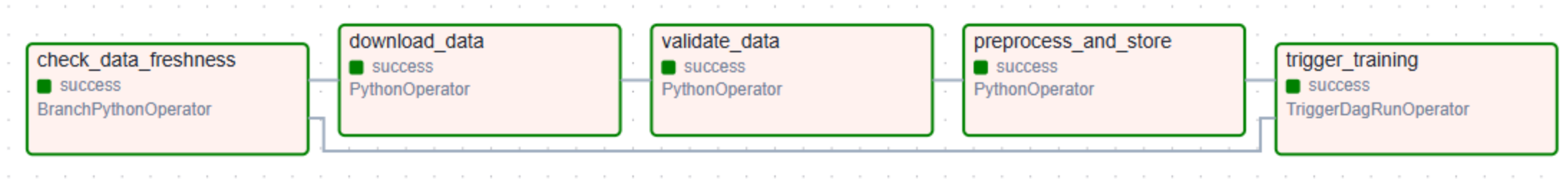
evaluate.py : calcule les métriques (MAE, R^2 , MAPE) sur le jeu de test après l'entraînement.

s3_storage.py

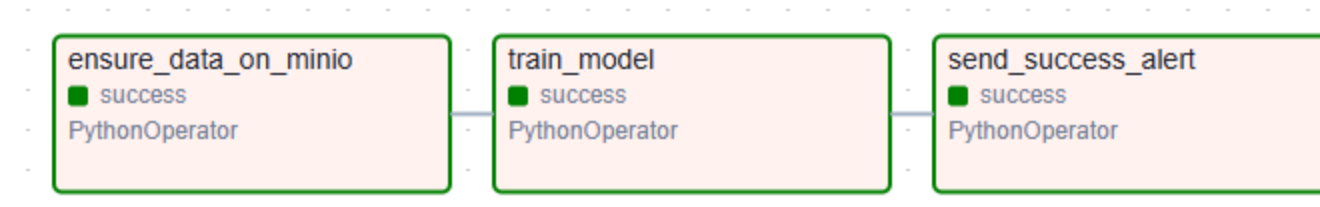
alerting.py

IV. PIPELINE D'ENTRAÎNEMENT (CONTINUOUS TRAINING)

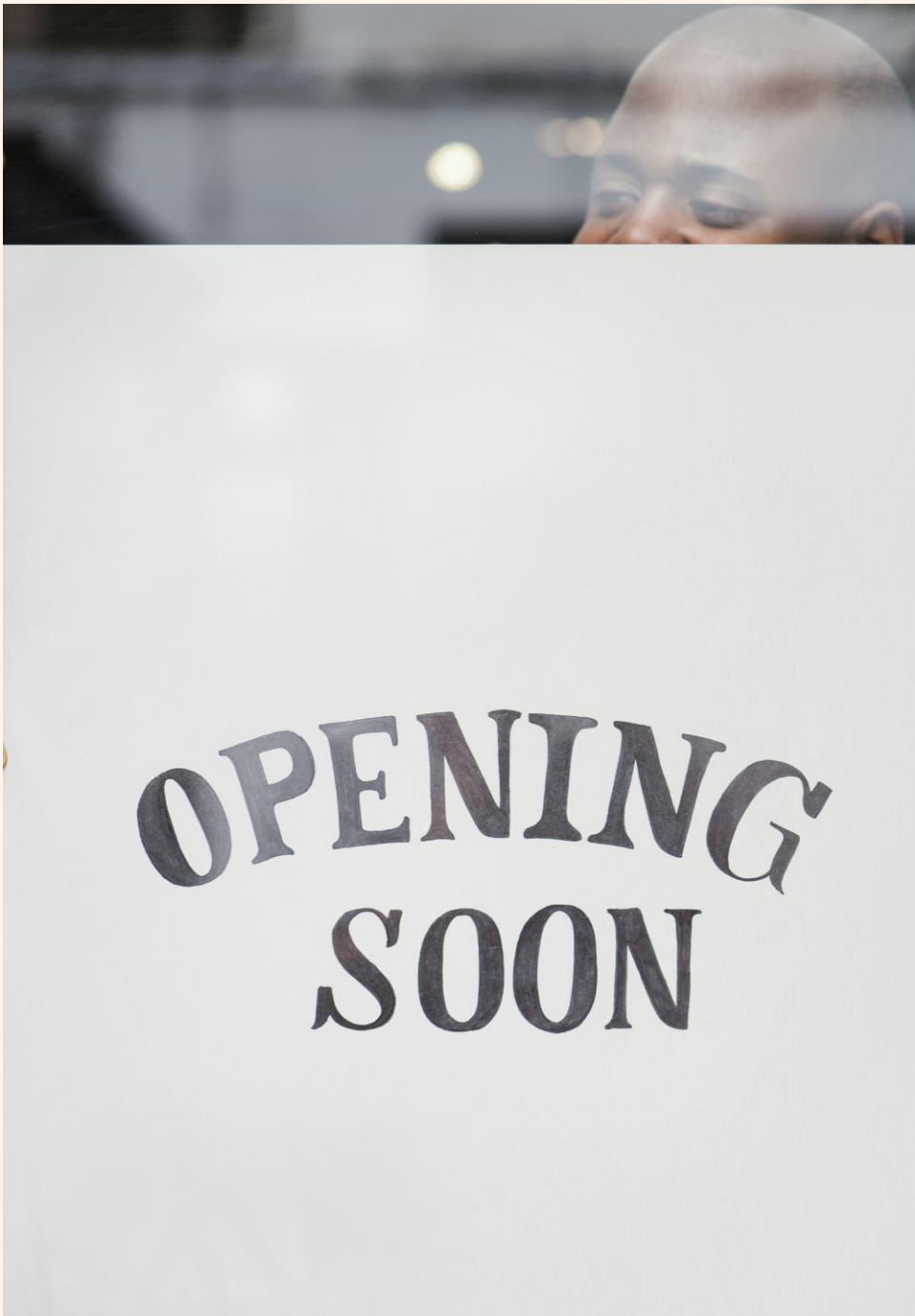
data_pipeline_dag.py



training_dag.py



V. SERVING : API & WEB APP

A photograph of a person's face, partially visible through a window. The person is looking down. The window has the text "OPENING SOON" written on it in a stylized, arched font. The background is a solid orange color.

**OPENING
SOON**

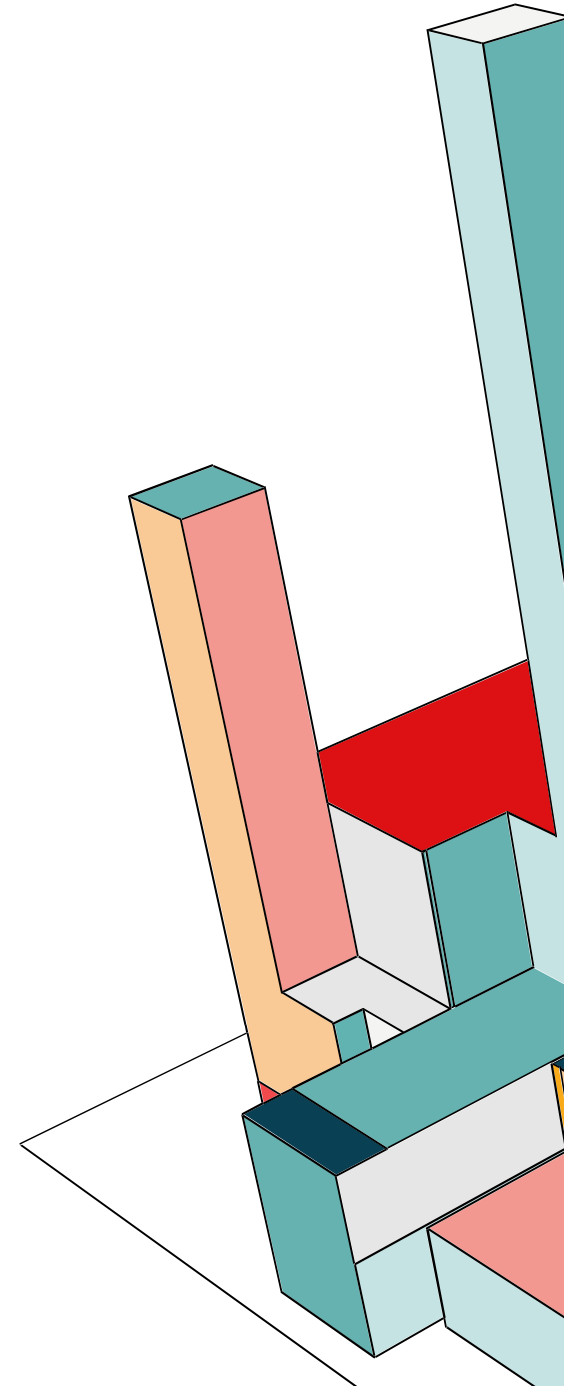
V. SERVING : API & WEB APP

FastAPI

- 4 endpoints (POST /predict, GET /health, GET /metrics, GET /docs)
- Au démarrage : charge le modèle depuis MLflow + artefacts preprocessing depuis MinIO

WebApp (l'interface graphique) :

- L'utilisateur sélectionne les caractéristiques.
- La webapp transmet les données à Fast API qui s'occupera de prétraiter les données (encoding zipcode, feature engineering) + prédire via le modèle + retourner le prix.
- La WebApp affichera ensuite le prix.



V. SERVING : API & WEB APP

Property Details

Bedrooms

3

012

Bathrooms

3.75

08

Floors

1.5

13.5

Living area (sqft)

2000

Lot size (sqft)

5000

Above ground (sqft)

1500

Basement (sqft)

500

Quality & Features

Condition (1-5)

4

15

Grade (1-13)

8

113

Waterfront

Does the property face the water?

0

1

View quality (0-4)

0

04

Construction

Year built

1990

Year renovated (0 = never)

0

Location

ZIP code

98103

Latitude

47.6516

Longitude

-122.348

Neighborhood

Avg neighbor living (sqft)

1800

Avg neighbor lot (sqft)

5000

Sale Date

Year of sale

2015

Month of sale

6

112

Estimate Price

Estimated Price: \$708,679

Raw value: \$708,679.38

King County, Washington

Example Properties

Click an example to fill in the form

Bedrooms	Bathrooms	Living area (sqft)	Lot size (sqft)	Floors	Waterfront	View quality (0-4)	Condition (1-5)	Grade (1-13)	Above ground (sqft)	Basement (sqft)	Year built	Year renovated (0 = never)	ZIP code	Latitude	Longitude	Avg neighbor li
3	2.5	2000	5000	2	0	0	3	8	1500	500	1990	0	98103	47.6516	-122.348	1800
4	3	3500	8000	2	1	4	5	11	2500	1000	2005	0	98039	47.6305	-122.24	3200
2	1	900	3000	1	0	0	3	6	900	0	1960	0	98178	47.5005	-122.236	1100

14

DATA713 ML Ops Project / King County House Sales Dataset / Model: XGBoost + Optuna / Tracking: ML flow



VI. DÉPLOIEMENT

VI. DÉPLOIEMENT

Stratégie d'Environnements :

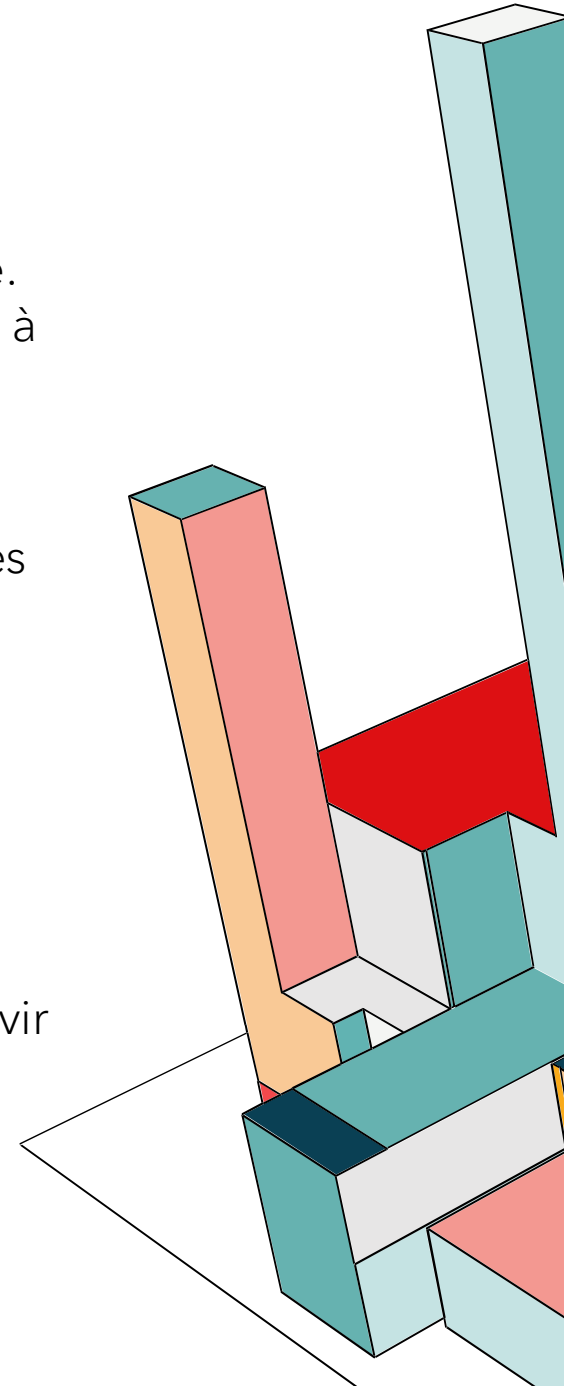
- Dev (Docker Compose) : Déploiement rapide et centralisé pour l'itération locale.
- Prod (Kubernetes) : Haute disponibilité via l'auto-réparation (Self-healing), mises à jour sans coupure (Rolling updates) et scalabilité horizontale de l'API.

Séparation des Responsabilités de Stockage :

- Data Lake (MinIO S3) : Stockage exclusif des données (brutes, pré-traitées) et des artefacts de Feature Engineering (encodeurs).
- Model Registry (MLflow) : Gestion centralisée des versions du modèle, des hyperparamètres et des métriques de performance.

Automatisation & Observabilité :

- Orchestration : Airflow gère les pipelines d'ingestion (vers MinIO) et d'entraînement (vers MLflow).
- Inférence : FastAPI fusionne les données (MinIO) et le modèle (MLflow) pour servir les requêtes.
- Monitoring : Suivi en temps réel des performances de l'infrastructure via Prometheus et Grafana.



VII. MONITORING



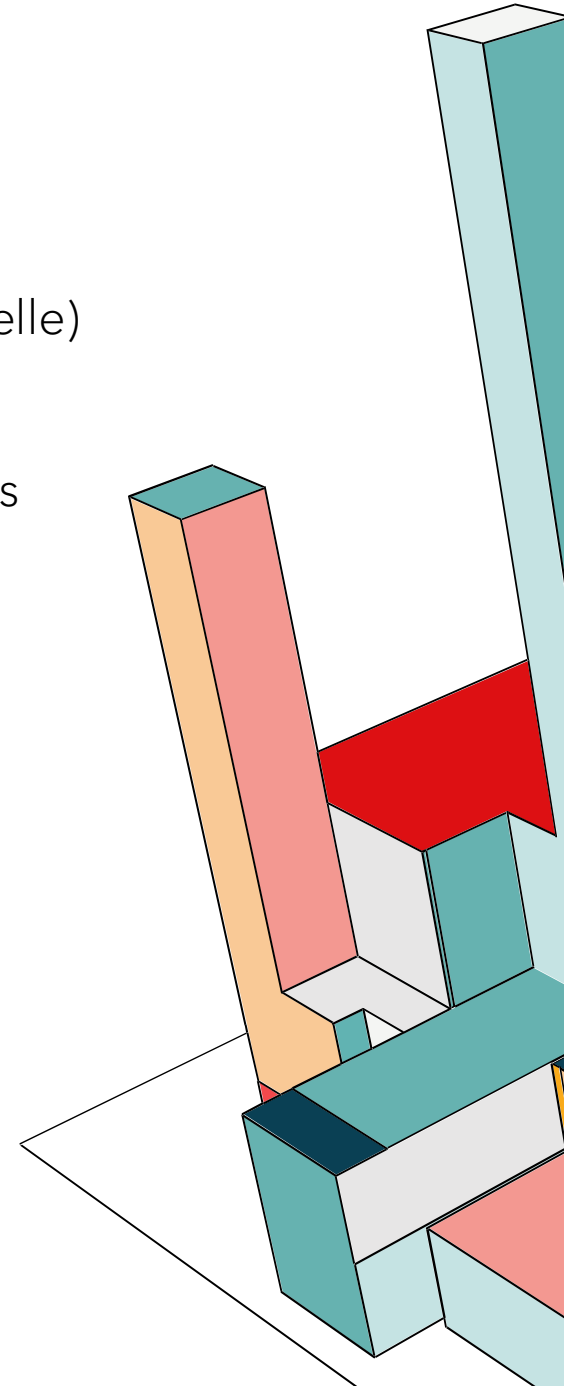
VII. MONITORING

Matériel :

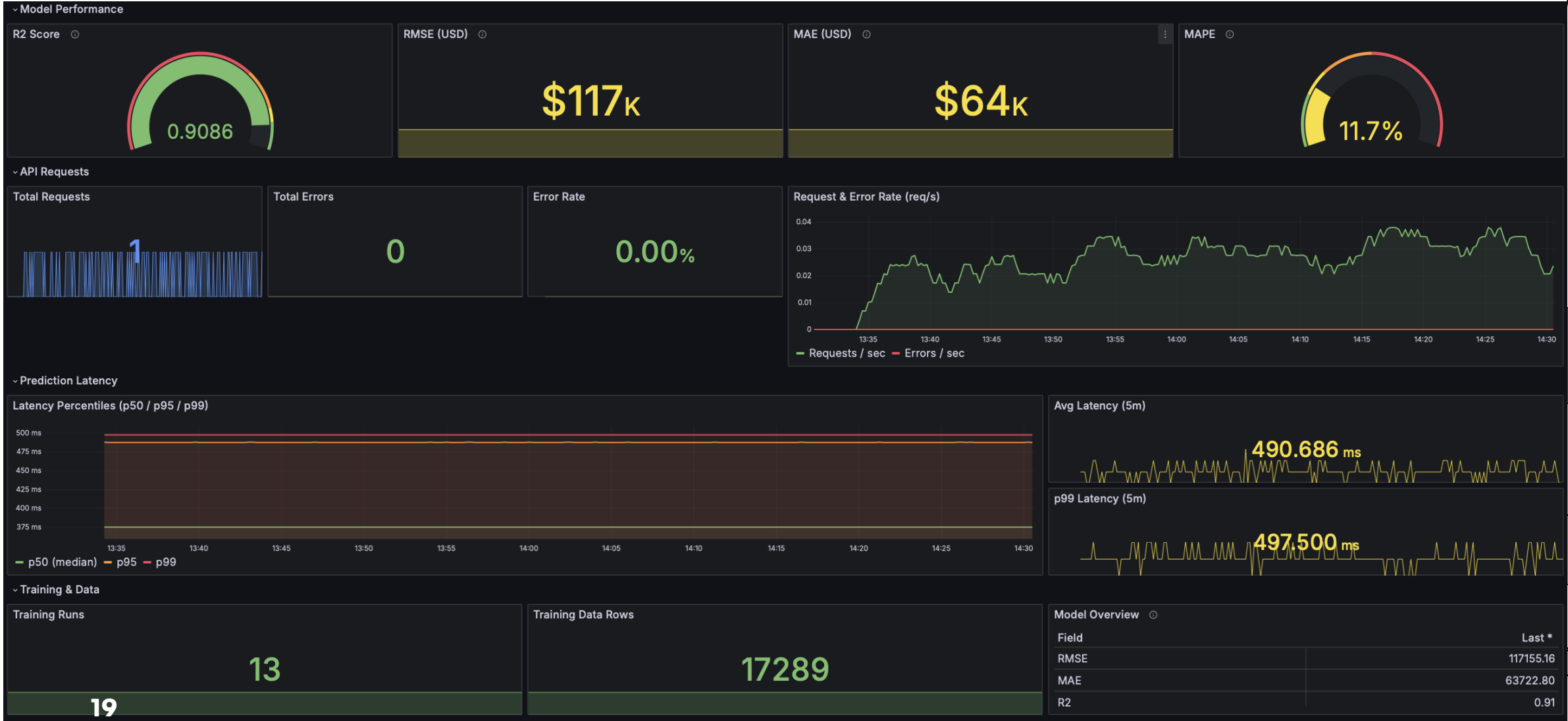
- Combinaison Prometheus (collecte/stockage métriques) + Grafana (interface visuelle)

Principes suivis :

- **Infrastructure as Code (IaC)** : Pas de configurations à la main. Les dashboards et les sources de données sont injectés via des **ConfigMaps**.
→ Si le pod tombe, il revient avec tout son paramétrage.
- **Persistance des données** : Grâce aux **PVC (Persistent Volume Claims)**, on conserve l'historique des prédictions même après un redémarrage du cluster.
→ On ne perd pas nos métriques.
- **Observabilité ML complète** : On ne surveille pas que la latence (Ops). Au sein de notre JSON de dashboard, on suit le **RMSE** et le **R²** en direct.



VII. MONITORING





VI. DÉMONSTRATION